Epiq: An Epistemic Database for Agent-Driven Research

September 22, 2026

Abstract

Research tables compress an uncertain process into apparently definite cells. A value may conceal conflicting sources, an unsuccessful search, a disputed interpretation, or a correction learned after the fact. Epiq is a local-first database that makes these distinctions explicit. Its primary records are entities, versioned typed questions, source fragments, and attributed claims; tables are projections of these records. Epiq combines evidence-required assertions, explicit disagreement and search outcomes, bitemporal reads, revisable schemas, and derivation lineage with a deterministic interface for research agents. An optional research layer separates source retrieval from model interpretation and supports staged acceptance. We describe the system's philosophy, projection semantics, and SQLite implementation, and report a focused execution of 161 existing tests. The contribution is a coherent database interface for preserving the process of inquiry: what was asserted, why, by whom, and under which interpretation. Provenance makes this process inspectable; it does not certify factual truth.

1. Introduction

Consider an agent assembling a comparison of software vendors. A blank in an ordinary comparison table could mean that the agent never investigated a feature, investigated it unsuccessfully, or found conflicting answers and omitted both. A populated cell can be equally ambiguous: a quoted price may be outdated, refer to another plan, or have been inferred from an unreliable source. Once these distinctions disappear, subsequent agents must either trust the table or repeat the investigation.

Epiq treats a research database as a durable record of inquiry. Its organizing abstraction is an assertion answering a typed question about an entity, supported by identifiable evidence. The familiar entity-by-question matrix remains the primary human interface, but its cells expose their supporting claims and research state. A fresh agent can inspect that state, identify useful work, and extend it through validated operations.

Three principles organize the design. First, preserve the grounds for an assertion alongside its value. Second, represent disagreement, ignorance, and revision explicitly. Third, separate the fallible production of research findings from deterministic storage and projection. Together, these principles let research accumulate across sessions without requiring every new participant to inherit the previous participant's context or judgment.

This is a systems design paper about the inspected implementation, rather than a claim of a new temporal or provenance calculus. The architectural contribution is the integration of these ideas into an agent-facing research workflow.

Appendix A supplies an executable CLI walkthrough. Its displayed commands are checked in CI against the implementation, including the resulting cell states and retained historical evidence.

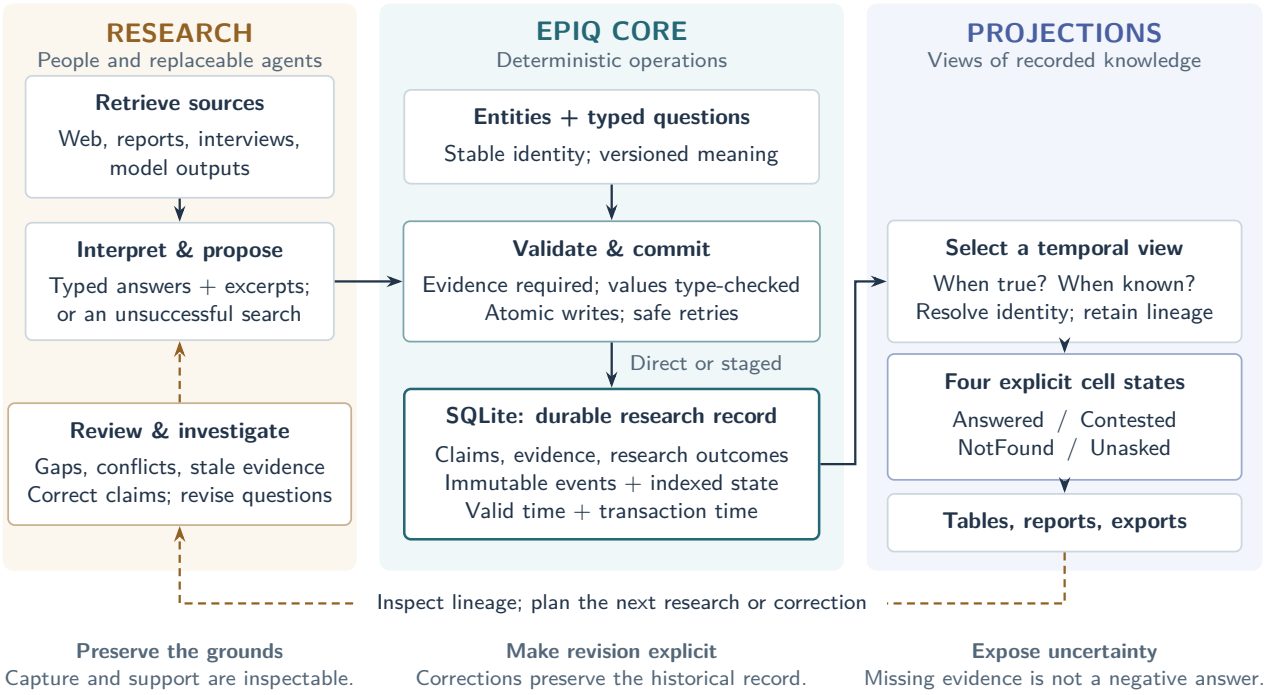


Figure 1: Epiq's research cycle and design principles. Fallible research supplies evidence and candidate answers to a deterministic core. Atomic writes preserve an audit history; temporal projections expose values, disagreement, and research outcomes with their lineage. The dashed feedback path initiates further investigation or explicit correction. Capture and review make the process inspectable without certifying truth.

2. Data Model and Epistemic Semantics

2.1 Questions define what a cell means

An **entity** has a stable identifier, kind, name, and optional attributes. A **question** declares a subject kind, value type, and interpretation metadata, including cardinality, research guidance, and temporal policy. A **source** identifies material and records metadata such as retrieval time; **evidence** is a bounded fragment used to support an assertion. A **claim** connects an entity and a particular question version to a typed value, evidence, confidence, temporal information, and an attributed creation event.

Questions are richer than column names. `supports_sso : Bool for Company` establishes an admissible answer space and an applicable population. Additional guidance can distinguish general availability from a feature offered only on an enterprise plan. This makes part of the research specification executable: malformed values and references to the wrong entity kind are rejected before publication.

Supported types include scalars, enumerations, dates and intervals, probabilities, distributions, structured JSON, entity references, and quantities whose unit is declared in the type. `Ref[Company]` forms a typed relationship; `Quantity[USD]` records the unit convention without implying a complete dimensional-analysis system. Single-valued and many-valued questions have different conflict semantics.

2.2 A cell summarizes assertions

Let $C(e, q; k, v)$ be the claims eligible for entity e and question q at transaction-time cutoff k and valid-time point v, after identity and visibility resolution. Let $U(C)$ contain their distinct canonically serialized values. The projection uses the following rules:

Condition	Cell state	Meaning
C is empty; no applicable completed unsuccessful search	`Unasked`	No eligible answer or recorded unsuccessful search
C is empty; an applicable unsuccessful search exists	`NotFound`	A bounded investigation produced no sufficient answer
C is nonempty; cardinality is one and $\lvert U(C)\rvert = 1$	`Answered`	Eligible claims agree on the value
Cardinality is one and $\lvert U(C)\rvert > 1$	`Contested`	Eligible claims contain different values
C is nonempty; cardinality is many	`Answered`	Multiple supported values are permitted

`NotFound` is an observation about research activity, not an assertion that a proposition is false. A supported Boolean `false` is an answer. Likewise, `Answered` does not mean independently verified or reviewed: it describes the relationship among currently eligible assertions. `Unasked` is a projection state, not proof that nobody ever investigated the cell; retracting all answers can leave it in that state if no applicable unsuccessful-search record remains.

For single-valued questions, a later conflicting assertion does not silently win. Both claims remain visible until an explicit correction changes their eligibility. Several claims can agree on one value while retaining separate evidence. Compatibility here is deliberately syntactic: the database does not infer that differently worded strings mean the same thing.

2.3 Evidence and confidence are separate concerns

Every asserted claim must cite existing evidence. Evidence may come from a web page, report, interview, personal record, or model output; non-web material need not acquire an artificial HTTP address. One claim can cite several fragments, and a fragment can support several claims. Structured locators can identify a page, section, or table.

Confidence is an explicit `low`, `medium`, or `high` annotation. It is distinct from a probability-valued answer, agreement among sources, and evidence freshness. The system does not automatically turn old evidence into lower confidence. A field's temporal policy instead exposes freshness as a separate signal. This prevents unrelated judgments from collapsing into a single apparently authoritative score.

3. Revision as Part of the Model

3.1 Two times, two kinds of correction

Epiq distinguishes when an assertion applies from when it entered the database's recorded knowledge. This follows the valid-time and transaction-time distinction in temporal database research. Snodgrass and Ahn, 1985.

Conceptually, an eligible claim satisfies

$$t_{\text{start}}(c) \le k < t_{\text{end}}(c), \qquad v_{\text{start}}(c) \le v < v_{\text{end}}(c; k),$$

where an absent endpoint is unbounded. The dependence of the valid-time endpoint on k matters: the database may learn only later that a fact ceased to hold. Identity, schema, evidence links, and other historical metadata also need the transaction-time cutoff; filtering claim timestamps alone is insufficient.

For example, a vendor's price is recorded as $20 on January 10. On March 5, research establishes that the price changed on February 1. Ending the old claim's validity on February 1 preserves the statement that it applied earlier. A query about February 10 made with a February 20 knowledge cutoff still reconstructs the earlier belief. Retraction has a different meaning: the assertion should no longer be accepted in the database's current account. Supersession atomically closes an old assertion and creates its replacement.

Timestamp inputs are compared as UTC instants, with inclusive starts and exclusive ends. Historical document imports can assign transaction times to complete phases, subject to chronological validation. These imports are

an explicit trusted operation; transaction timestamps should not be interpreted as independently authenticated evidence of when an external event occurred.

3.2 The schema is also open to criticism

Research can reveal that the question itself is wrong. "Has a spinnaker?" might conflate whether a boat design permits one with whether a particular boat carries one. Epiq records question challenges, including type, cardinality, population, unit, temporal, and definition problems. A challenge can carry an explanation, evidence, and a proposed replacement.

Question definitions are versioned. Explicit evolution can replace, refine, or split a question, atomically recording successor definitions, lineage edges, and predecessor retirement. Old claims keep their original question identifiers. Splitting the spinnaker question creates new research obligations; it does not manufacture answers by guessing how ambiguous old values should map to the new categories.

Entities similarly support aliases, merge redirects, and reversible retirement. Merging duplicate rows preserves original claim subject identifiers while current projections resolve them to a surviving identity. These mechanisms express a broader principle: changes in interpretation should remain observable rather than rewriting the record that produced them.

4. Deterministic Storage, Replaceable Research

4.1 An audit history within SQLite

The current implementation uses Python and SQLite, with no runtime dependencies for the core package. Its SQLite database is canonical. Earlier design documents proposed a JSONL ledger with disposable replay caches; that is not the present storage architecture.

Domain operations append typed events and update relational state within the same transaction. SQLite write-ahead logging and `BEGIN IMMEDIATE` serialize competing writers. Matrix reads execute within a read transaction and load claims and lineage in batches, avoiding a separate query for every cell. Triggers prohibit updates and deletions of events, sources, evidence, and retained capture artifacts.

Append-only history does not mean that every SQL row is immutable. Claim closure updates status and transaction endpoints while appending its event; visibility, proposal decisions, and background-job progress also use mutable state. The audit history and indexed state coexist. The implementation should therefore not be described as a pure replay engine with a separately complete event-only recovery format.

Atomic batches can create evidence and multiple claims using local symbolic references. A malformed final operation rolls back the entire batch, including events. A higher-level `record` command packages this common write pattern. Staged documents run validation without exposing candidate entities, schemas, or answers; approval revalidates against current state and publishes the bundle atomically.

Retry semantics are equally important for agents. Identical normalized evidence and claim writes reuse existing records; supported document and proposal operations also maintain durable request receipts. Actor-scoped keys distinguish retries from new attempts. Because assertion deduplication does not make actor identity a measurement dimension, repeated ratings or model runs should be represented as separate observation entities with compound identities.

4.2 Provenance has several layers

The optional source pipeline distinguishes retrieval from interpretation. A provider produces a bounded source bundle; a reasoner selects supplied sources and returns typed findings with supporting quotations. URLs, hashes, and capture metadata come from the bundle rather than model-generated citations. Exact quote checks establish that the submitted support appears in the captured material.

The resulting provenance has three separate layers: capture metadata and excerpt matching; supporting spans linking evidence to claims; and explicit claim reviews with reviewer, reason, and event time. Full source text can optionally be retained as an immutable content-addressed artifact. Manual and legacy evidence remain usable but carry an explicit unverified status when capture verification is unavailable.

These distinctions prevent a common category error. A matching quote establishes textual traceability, not entailment, source independence, credibility, or truth. A review is an attributed action, not authenticated proof of a human identity. Rejected reviews and adverse evidence assessments are visible annotations; they do not automatically retract every affected assertion. Structural integrity and provenance diagnostics are reported separately.

4.3 Agents participate through a protocol

The CLI emits versioned JSON envelopes with data, warnings, errors, and suggested next steps. Capability discovery, schemas, compact context, gap detection, contradiction reports, and refresh plans let agents orient themselves and identify work without guessing command syntax. Suggested actions include exact argument arrays and metadata describing effects.

The deterministic core performs neither web retrieval nor model inference. An optional web application orchestrates replaceable research workers, persists job requests and checkpoints, and commits completed cell findings with their progress records. Interrupted work is recovered as an explicit failure or cancellation rather than silently relaunched. The spreadsheet, CLI, and Python API share the underlying store.

5. Computation and Human Interfaces

Epiq also records derived claims. Arithmetic operations, weighted averages, bounded formulas, and relationship-based propagation retain their input claims and inherited evidence. Dependencies distinguish operands, parameters such as sourced weights, and relationship paths. When an input becomes inactive or a newer claim becomes available, stale-derivation diagnostics identify affected results. Recalculation is explicit, preserving the original calculation as a historical assertion.

Distribution-valued questions preserve uncertainty without requiring an immediate scalar summary. For example, several weather forecasts can become an empirical or weighted empirical distribution with source lineage. Categorical distributions validate the declared outcome set and probability mass. Storing these objects does not imply calibrated uncertainty, independent samples, or automatic Bayesian inference.

The interface exposes this model through a spreadsheet with cell-level evidence inspection, conflict states, schema challenges, research actions, and calculated fields. A cell edit creates an assertion rather than silently replacing the past. HTML, Excel, and long-form claim exports carry lineage into downstream analysis. Snapshot backups and checksummed project bundles provide portable copies of the database and retained artifacts.

6. Validation, Positioning, and Limits

We inspected the current storage, projection, temporal, provenance, proposal, API, and language modules, and executed eight existing test modules: `test_store`, `test_distributions`, `test_temporal`, `test_change_sets`, `test_provenance`, `test_hardening`, `test_public_api`, and `test_dsl`. All **161 tests passed**. The suite exercised conflicting claims, atomic rollback, historical cutoffs, retry receipts, staged acceptance, provenance checks, concurrent writes, migration recovery, and snapshot consistency. A projection test also verified bounded SQL statement counts on a 100-row, ten-question fixture. These results establish regression coverage for specific behaviors, not a throughput benchmark, complete correctness proof, or measurement of research quality. Live source services and the full browser suite were not exercised for this paper.

Epiq combines established ideas with a particular interaction model. Temporal databases motivate its two time axes. Provenance research gives a formal treatment of how query results depend on inputs; Epiq implements concrete claim, evidence, and derivation links rather than a general semiring query algebra. Green, Karvounarakis, and Tannen, 2007. Its locally owned project files and offline core align with local-first software's emphasis on user control, although it does not implement distributed synchronization or the full collaborative vision of that work. Kleppmann et al., 2019.

Several limits follow from these choices. SQLite serializes writers; large research corpora and high write concurrency require evaluation beyond the fixtures used here. The web application targets trusted local use and lacks multi-tenant authentication. Claim conflicts are detected by value equality, leaving semantic reconciliation to researchers. Current projections select the latest question definition by name while collecting claims across its versions; versioning preserves attribution but is not a general automatic conversion system for incompatible schemas. EpiQL

remains a narrow parser for question declarations and count-over-filter derivations, not a complete executable research language.

A useful future evaluation would compare Epiq with an ordinary spreadsheet and a citation-bearing table on the same research tasks. Measures should include recovery of source lineage, correction propagation, repeated work across agent sessions, detection of ambiguous questions, and time required for human review. Such a study could test the system's central hypothesis: making the state of inquiry explicit improves the reliability and continuity of research.

7. Conclusion

Epiq makes the evidence-bearing assertion the durable unit of research and the table a revisable interpretation of those assertions. Its design preserves distinctions that flattened research outputs commonly lose: disagreement versus multiplicity, unsuccessful search versus negation, changed facts versus corrected beliefs, and sound data structure versus credible evidence. The result is an inspectable interface through which agents and people can extend, challenge, and reuse research without erasing how it was produced.

References

1. Richard Snodgrass and Ilsoo Ahn. 1985. A Taxonomy of Time in Databases. *Proceedings of ACM SIGMOD*, 236–246.
2. Todd J. Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance Semirings. *Proceedings of ACM PODS*, 31–40.
3. Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-First Software: You Own Your Data, in Spite of the Cloud. *Proceedings of ACM Onward!*, 154–178.

Implementation Note

This paper describes the local Epiq 0.1.0 working tree inspected on September 22, 2026: base commit `308768e`, including existing uncommitted changes through storage schema 15. It is not a description of that commit alone or a claim about a published release. Principal implementation sources are storage, projections, and the public API. The inspected working tree also includes `temporal.py`, `change_sets.py`, and `provenance.py`, plus notes on historical imports and source providers; these additions are outside this paper's documentation commit. Original parent-directory briefs were treated as design history wherever they differed from executable behavior. The executable appendix uses the committed CLI surface and is checked independently of those local additions.

The architecture figure is editable TikZ source. Rebuild the vector figure and paper with `bash docs/build-short-paper.sh` from the repository root; the build requires Pandoc and a LaTeX installation with TikZ.

Appendix A. An Executable Research Cycle

This maintained example follows one synthetic company and one Boolean question: does Acorn support SSO? All source excerpts and dates are fixtures; no network request or model call is made. The purpose is to reproduce the database's behavior, not establish a fact about a real company.

Install the checkout with `uv sync --extra test`, then run `source .venv/bin/activate`. Execute the following Bash snippets in order in the same shell. The first snippet selects a fresh temporary database through `EPIQ_DB`, leaving the workspace's saved selection unchanged. The example checker supplies `EPIQ_EXAMPLE_DB` during automated runs; an interactive run can leave it unset.

A.1 Define the entity and question

```
export EPIQ_DB="${EPIQ_EXAMPLE_DB:-$(mktemp -d)/research.sqlite}"
epiq init --name "Paper: an epistemic research cycle"
epiq entity Company Acorn
epiq question supports_sso --for Company --type Bool
epiq --format table matrix --kind Company
```

The cell is `Unasked`. Defining a column establishes an answer space, but supplies no answer. Rerunning the setup against an existing database fails at `init`; it does not overwrite the project.

A.2 Record an unsuccessful search

```
epiq --actor agent:example not-found \
  --subject Acorn --question supports_sso \
  --query "Acorn security documentation, checked for SSO" \
  --notes "The inspected overview did not address SSO."
epiq --format table matrix --kind Company
```

The cell is now `NotFound`. This records a bounded research outcome, not the Boolean value `false`.

A.3 Add evidence and a supported assertion

```
paper_yes=$(epiq --actor agent:example --select evidence_id evidence \
  --url https://example.test/acorn/security \
  --title "Acorn security guide (synthetic)" \
  --retrieved-at 2026-01-10 \
  --excerpt 'Acorn includes SSO on every plan from January 1, 2026.' |
  python3 -c 'import json,sys; print(json.load(sys.stdin))')
epiq --actor agent:example assert \
  --subject Acorn --question supports_sso --value true \
  --valid-from 2026-01-01 --evidence "$paper_yes"
epiq --format table matrix --kind Company
```

The cell is `Answered` with value `true`. `--select evidence_id` returns a JSON string; the small Python filter decodes it before passing the identifier to the next command. The valid date comes from the fixture; the transaction timestamp is the time this command runs. The earlier unsuccessful search remains in history. These manually submitted excerpts have not been independently capture-verified or reviewed.

A.4 Preserve disagreement

```
paper_no=$(epiq --actor agent:example --select evidence_id evidence \
  --url https://example.test/comparison/acorn \
  --title "Comparison page (synthetic)" --retrieved-at 2026-01-11 \
  --excerpt 'As of January 1, 2026, Acorn does not support SSO.' |
  python3 -c 'import json,sys; print(json.load(sys.stdin))')
paper_conflict=$(epiq --actor agent:example --select claim_id assert \
  --subject Acorn --question supports_sso --value false \
  --valid-from 2026-01-01 --evidence "$paper_no" |
  python3 -c 'import json,sys; print(json.load(sys.stdin))')
epiq --format table matrix --kind Company
paper_known_at=$(epiq history | python3 -c \
  'import json,sys; print(json.load(sys.stdin)["data"][-1]["recorded_at"])')
```

The cell is `Contested`: two eligible claims give different answers to a single-valued question. Recording the second claim does not replace the first. The saved cutoff is the actual event timestamp, allowing the next step to reconstruct this state without hard-coding the date of execution.

A.5 Correct explicitly, then reconstruct the earlier belief

Suppose a reviewer establishes that the comparison page described a different product with the same name. That judgment is supplied by the reviewer; Epiq does not infer it from the excerpts.

```
epiq --actor human:reviewer retract "$paper_conflict" \
  --reason "The comparison described another product named Acorn."
epiq --format table matrix --kind Company
epiq --format table matrix --kind Company --known-at "$paper_known_at"
epiq doctor
```

The current cell returns to `Answered` with value `true`; the historical read remains `Contested`. Both source fragments and the retracted assertion remain addressable. `doctor` checks structural integrity and reports provenance gaps separately; it does not certify the surviving answer.

Step	Current state	Eligible values
Define the question	`Unasked`	None
Record the unsuccessful search	`NotFound`	None
Assert the security-guide answer	`Answered`	`true`
Assert the conflicting answer	`Contested`	`true`, `false`
Retract the mistaken interpretation	`Answered`	`true`
Read at the saved pre-retraction cutoff	`Contested`	`true`, `false`

A.6 Maintenance contract

The five marked snippets above are the executable source of this example. The normal Markdown-example check runs them together in a temporary workspace. A dedicated regression test checks each transition, evidence and actor lineage, retention of the unsuccessful-search event, and the historical read after retraction. This checks semantics as well as command syntax. To validate this appendix from the repository root:

```
python scripts/check_markdown_examples.py docs/epiq-short-paper.md
python -m pytest -q tests/test_paper_example.py
```

The checker removes its temporary project after the run. An interactive run retains the database at `$EPIQ_DB` for further inspection; `epiq history` and `epiq dossier Acorn` expose the complete record.